Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Date of Performance: 10/10/2026 **Date of Submission: 10/10/2026**

Class: T1

Student Roll No: 2401064 **Student Name: Kushagra Mehta**

Experiment 9: Infrastructure as Code Using Terraform on AWS

Aim:

To implement Infrastructure as Code (IaC) using Terraform for provisioning and managing AWS cloud infrastructure.

This experiment maps to LO4, LO5 and LO6.

LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

LO5: Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.

LO6: Develop and demonstrate an end-to-end DevOps pipeline mini-project integrating all stages of DevOps from code commit to deployment and monitoring.

Theory:

Cloud computing is a technology that provides computing resources such as servers, storage, networking, and databases over the internet. It allows users and organizations to access IT infrastructure without purchasing and maintaining physical hardware. Amazon Web Services (AWS) is one of the widely used cloud computing platforms that provides various services for building, deploying, and managing applications.

Terraform is an open-source Infrastructure as Code (IaC) tool developed by HashiCorp. It is used to create, manage, and provision infrastructure resources through configuration files instead of configuring them manually. Terraform supports multiple cloud platforms, including AWS, Microsoft Azure, and Google Cloud Platform.

Infrastructure as Code is an approach in which infrastructure is defined and managed using machine-readable configuration files. It helps automate infrastructure deployment and makes the process consistent, repeatable, and easier to maintain. With IaC, the same configuration can be reused to create similar infrastructure environments whenever required.

Terraform uses the HashiCorp Configuration Language (HCL) to define infrastructure resources. A Terraform configuration generally contains provider, resource, and output blocks.

Provider: A provider is a plugin that allows Terraform to communicate with a cloud platform or service. In this experiment, the AWS provider is used to connect Terraform with Amazon Web Services.

Resource: A resource block defines the infrastructure component that Terraform should create or manage. In this experiment, the aws_instance resource is used to provision an Amazon EC2 instance.

Output: An output block displays important information about the created infrastructure, such as the EC2 instance ID, public IP address, and instance type.

Amazon Elastic Compute Cloud (EC2) is an AWS service that provides scalable virtual computing servers in the cloud. An EC2 instance acts like a virtual machine on which users can install operating systems, run applications, and perform computing tasks. While launching an EC2 instance, parameters such as the Amazon Machine Image (AMI), instance type, and region must be specified.

An Amazon Machine Image (AMI) is a template that provides the operating system and other required software configuration for an EC2 instance. The instance type determines the computing capacity, including processor, memory, and networking capabilities. In this experiment, an Amazon Linux 2023 AMI is used to launch a t3.micro EC2 instance in the AWS Mumbai region (ap-south-1).

Terraform follows a workflow to manage infrastructure. First, the terraform init command initializes the working directory and downloads the required provider plugins. The terraform validate command checks the configuration for errors. The terraform plan command previews the changes Terraform intends to make. The terraform apply command creates or updates the infrastructure according to the configuration. The terraform output command displays the output values defined in the configuration.

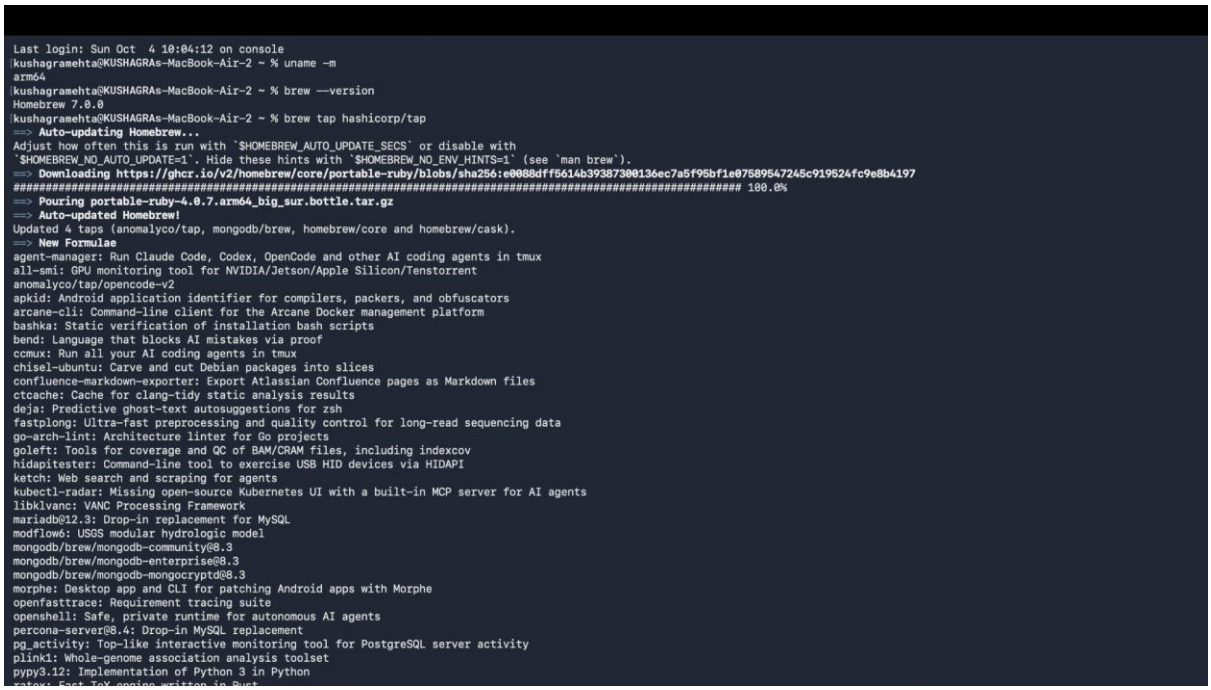
Terraform also maintains a state file, which records information about the infrastructure managed by Terraform. This state helps Terraform identify existing resources and determine what changes are required to match the desired configuration.

Finally, the terraform destroy command removes the infrastructure resources managed by the configuration. In this experiment, it is used to terminate the EC2 instance after verification, helping prevent unnecessary resource usage and potential charges.

Implementation:

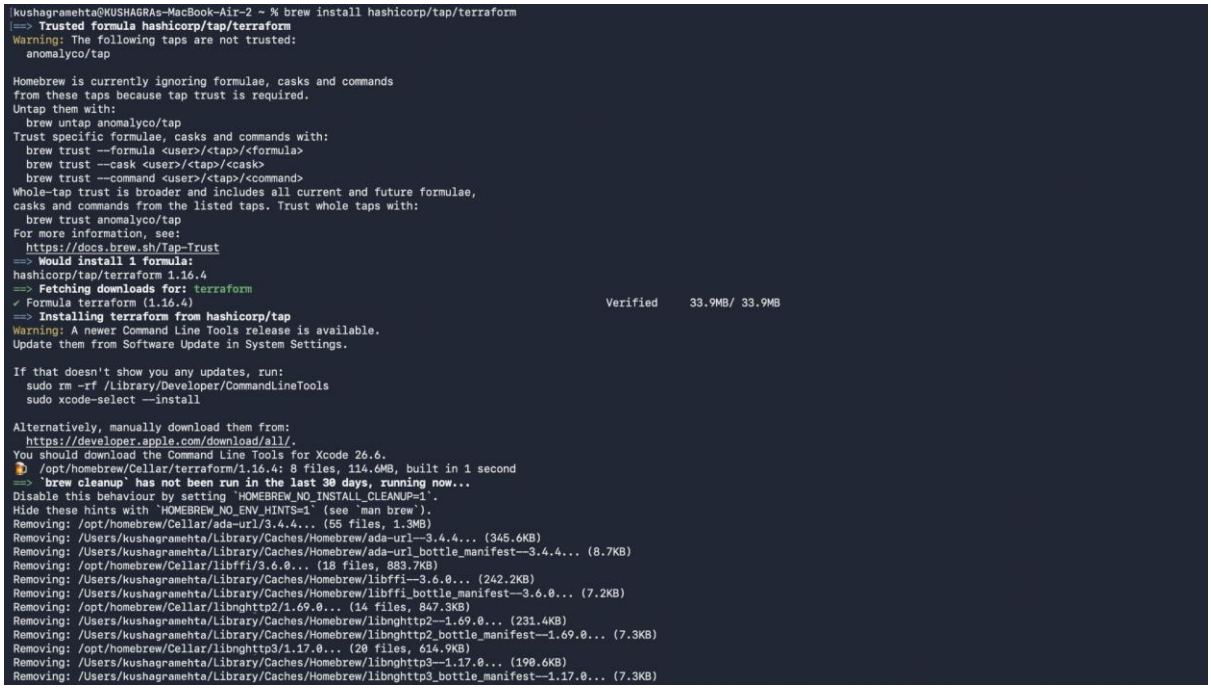
1. Install Homebrew:

Install Homebrew on the macOS system to manage software packages and dependencies.



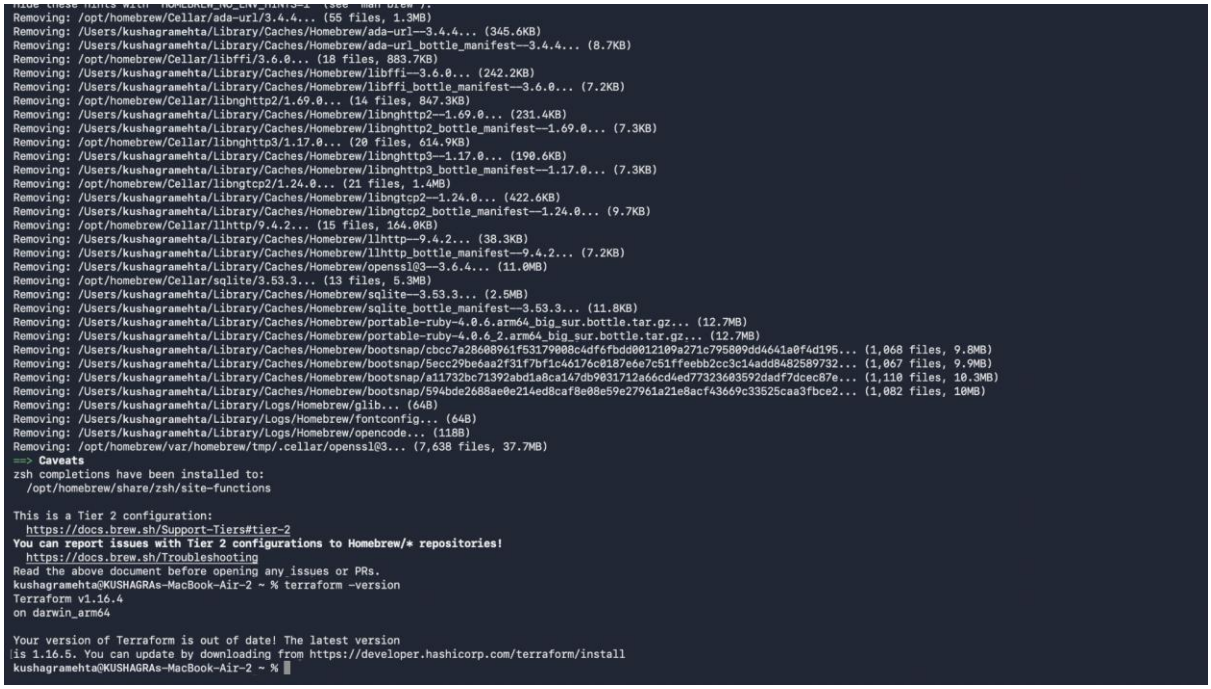
2. Install Terraform:

Tap the HashiCorp repository and install Terraform using Homebrew.



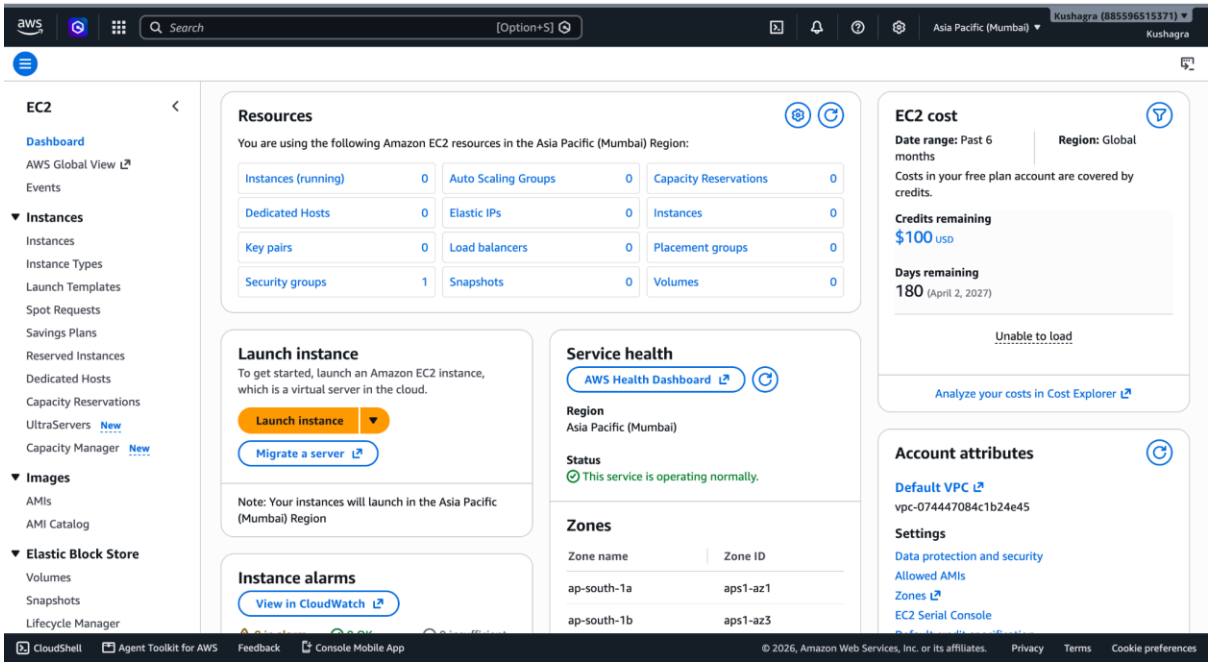
3. Verify the Terraform Installation:

Verify the installation of Terraform and check its installed version.



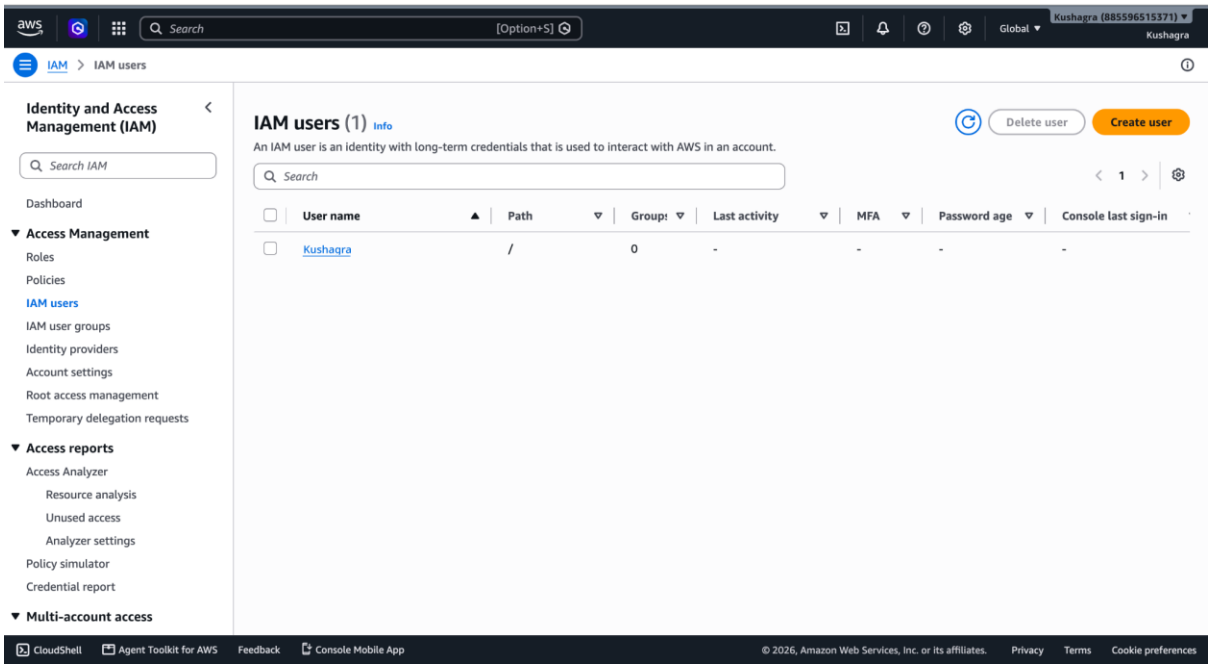
4. Open the AWS Management Console:

Access the AWS Management Console and view the initial EC2 dashboard state.



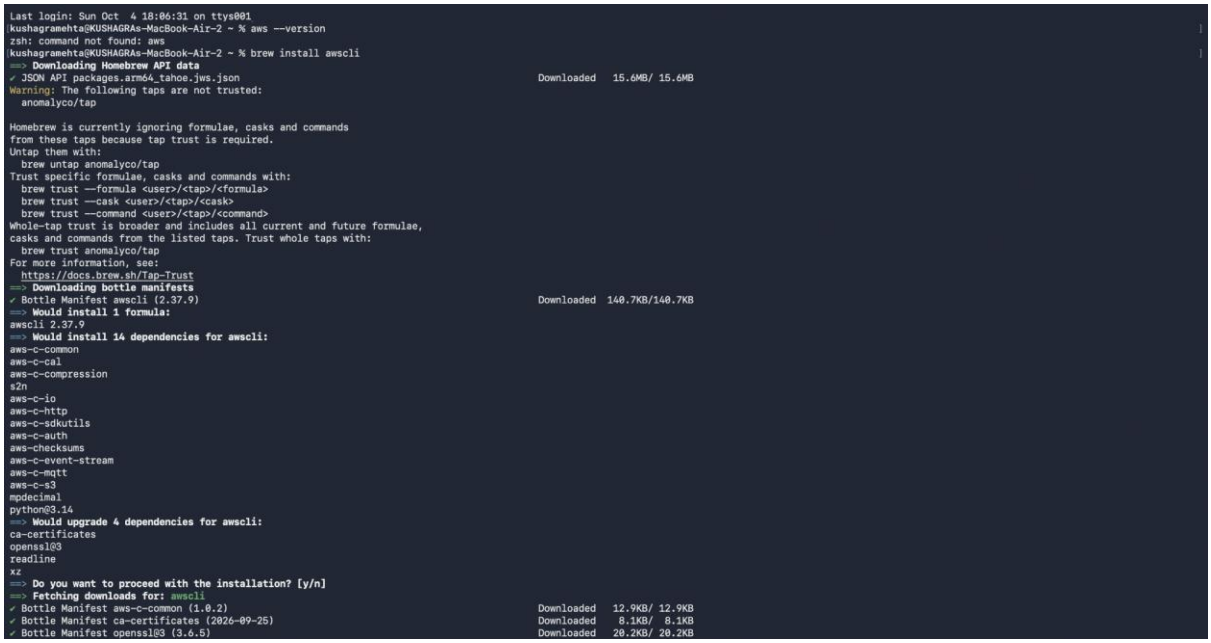
5. Navigate to IAM Users:

Navigate to IAM Users in the AWS Management Console to manage access credentials.



6. Install the AWS CLI:

Install the AWS Command Line Interface (AWS CLI) using Homebrew.



7. Verify the AWS CLI Installation:

Verify the successful installation of the AWS CLI version.

```
/opt/homebrew/share/awscli/examples
[kushagramehta@KUSHAGRAs-MacBook-Air-2 ~ % aws --version
 aws-cli/2.37.9 Python/3.14.8 Darwin/25.4.0 source/arm64
 kushagramehta@KUSHAGRAs-MacBook-Air-2 ~ %
```

8. Configure the AWS CLI:

Configure the AWS CLI with Access Key ID, Secret Access Key, and default region settings using aws configure, then verify the AWS identity using aws sts get-caller-identity.

```
[kushagramehta@KUSHAGRAs-MacBook-Air-2 ~ % aws sts get-caller-identity
{
    "UserId": "AIDA44MNKNQVXLJOMZNW3",
    "Account": "885596515371",
    "Arn": "arn:aws:iam::885596515371:user/Kushagra"
}
```

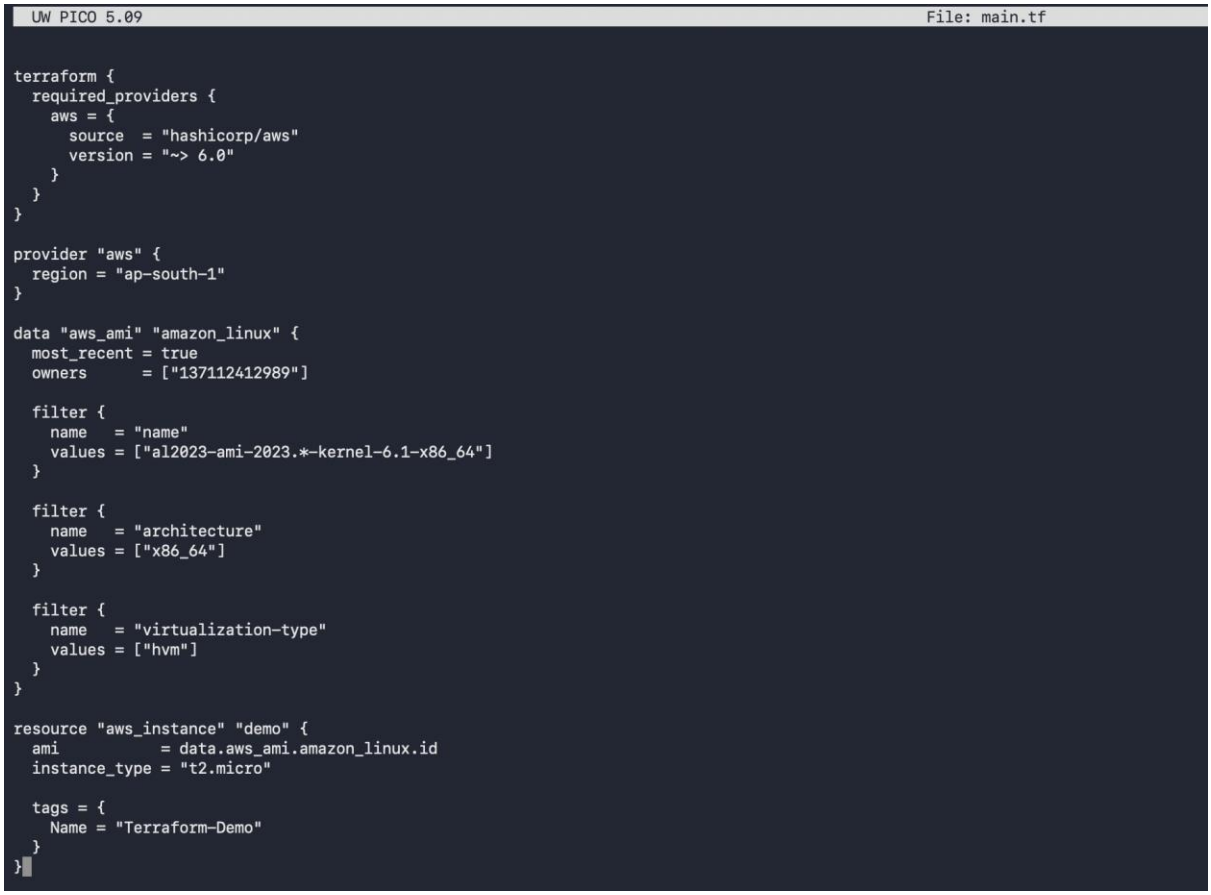
9. Create the Working Directory:

Create a working directory for the Terraform configuration files and navigate into it.

```
[kushagramehta@KUSHAGRAs-MacBook-Air-2 ~ % cd ~/Terraform
 cd: no such file or directory: /Users/kushagramehta/Terraform
[kushagramehta@KUSHAGRAs-MacBook-Air-2 ~ % mkdir -p ~/Terraform
[kushagramehta@KUSHAGRAs-MacBook-Air-2 ~ % cd ~/Terraform
[kushagramehta@KUSHAGRAs-MacBook-Air-2 Terraform % pwd
 /Users/kushagramehta/Terraform
[kushagramehta@KUSHAGRAs-MacBook-Air-2 Terraform % nano main.tf
```

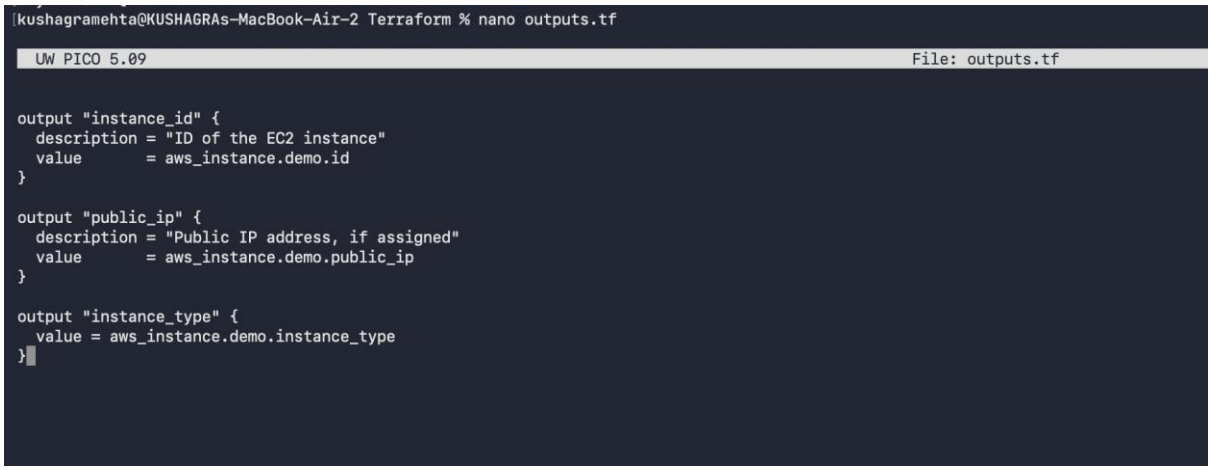
10. Write the main.tf Configuration:

Create and write the main.tf configuration file specifying the AWS provider, AMI data source, and EC2 instance resource.



11. Define the Outputs:

Define output variables in outputs.tf to display instance details such as instance ID, public IP, and instance type.



12. Initialize Terraform:

Initialize the working directory and install required provider plugins using terraform init.

```
kushagramehta@KUSHAGRAs-MacBook-Air-2 Terraform % ls
main.tf         outputs.tf
kushagramehta@KUSHAGRAs-MacBook-Air-2 Terraform % terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.67.0...
- Installed hashicorp/aws v6.67.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
kushagramehta@KUSHAGRAs-MacBook-Air-2 Terraform %
```

13. Validate the Configuration:

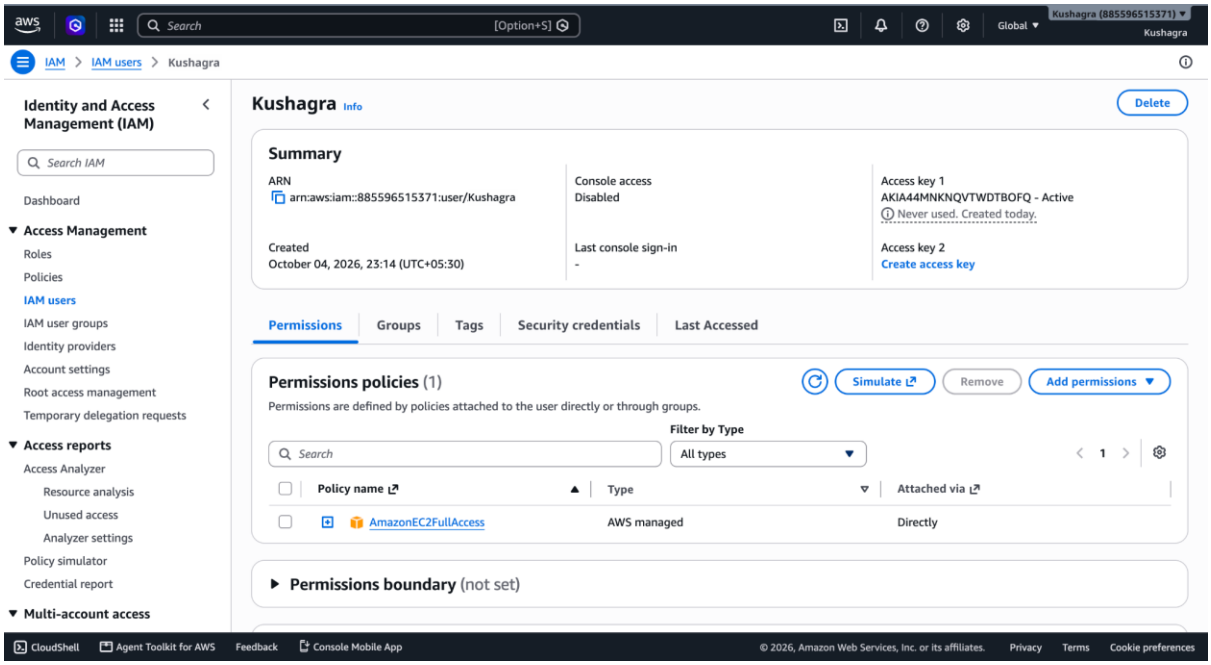
Validate the Terraform configuration files for syntax correctness using terraform validate.

```
kushagramehta@KUSHAGRAs-MacBook-Air-2 Terraform % terraform validate
Success! The configuration is valid.

kushagramehta@KUSHAGRAs-MacBook-Air-2 Terraform %
```

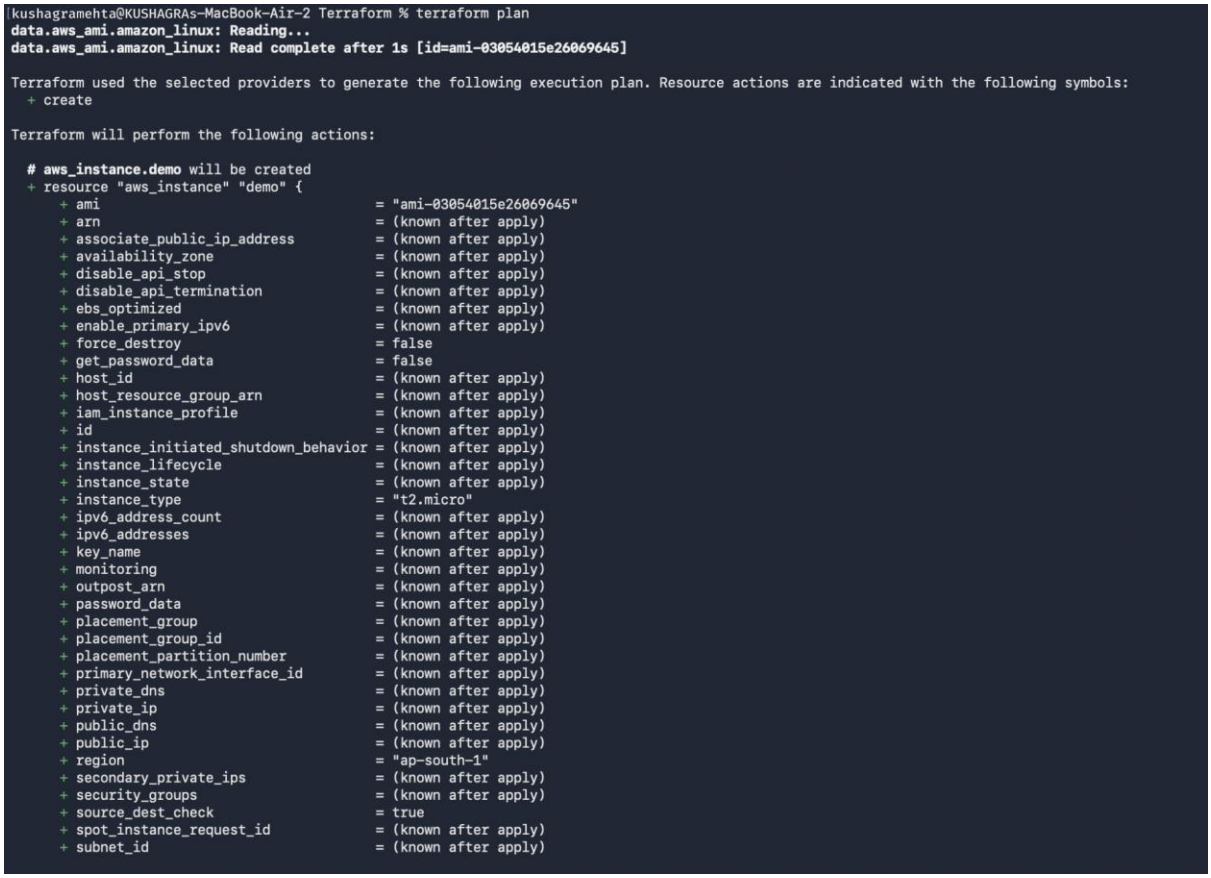
14. Review IAM Permissions:

Review permissions and access policies attached to the IAM user in AWS Console.



15. Generate the Execution Plan:

Generate and review an execution plan using terraform plan to see planned infrastructure changes.



```
kushagramehta@KUSHAGRAs-MacBook-Air-2 Terraform % terraform plan
data.aws_ami.amazon_linux: Reading...
data.aws_ami.amazon_linux: Read complete after 1s [id=ami-03054015e26069645]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.demo will be created
  + resource "aws_instance" "demo" {
      + ami                                  = "ami-03054015e26069645"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + region                               = "ap-south-1"
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
      + subnet_id                            = (known after apply)
```

16. Apply the Configuration:

Apply the configuration using terraform apply to provision the EC2 instance on AWS.

```
kushagramehta@KUSHAGRAs-MacBook-Air-2 Terraform % terraform apply
data.aws_ami.amazon_linux: Reading...
data.aws_ami.amazon_linux: Read complete after 0s [id=ami-03054015e26069645]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.demo will be created
  + resource "aws_instance" "demo" {
      + ami                                  = "ami-03054015e26069645"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + region                               = "ap-south-1"
      + secondary_private_ips                = (known after apply)
```

```
      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + primary_network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)

      + secondary_network_interface (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_id   = (known after apply)
  ~ instance_type = "t2.micro" -> "t3.micro"
  + public_ip     = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.demo: Creating...
aws_instance.demo: Still creating... [00m10s elapsed]
aws_instance.demo: Creation complete after 13s [id=i-0719e36443ddc768f]

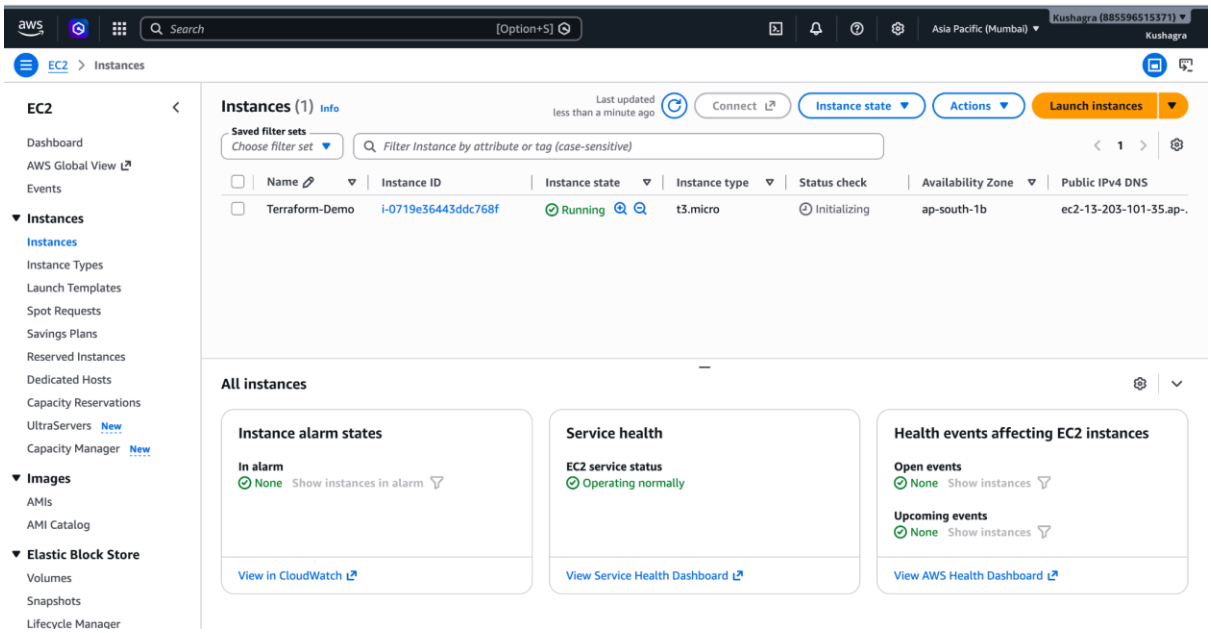
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-0719e36443ddc768f"
instance_type = "t3.micro"
public_ip = "13.203.101.35"
```

17. Verify the EC2 Instance:

Verify the newly provisioned EC2 instance running in the AWS Management Console.



18. Display the Outputs:

Query and display output variables using terraform output.

```
kushagramehta@KUSHAGRAs-MacBook-Air-2 Terraform % terraform output
instance_id = "i-0719e36443ddc768f"
instance_type = "t3.micro"
public_ip = "13.203.101.35"
kushagramehta@KUSHAGRAs-MacBook-Air-2 Terraform %
```

19. Destroy the Infrastructure:

Destroy the created infrastructure using terraform destroy to clean up resources.

```
kushagramehta@KUSHAGRAs-MacBook-Air-2 Terraform % terraform destroy
data.aws_ami.amazon_linux: Reading...
data.aws_ami.amazon_linux: Read complete after 2s [id=ami-03054015e26069645]
aws_instance.demo: Refreshing state... [id=i-0719e36443ddc768f]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

  # aws_instance.demo will be destroyed
  - resource "aws_instance" "demo" {
      - ami                                  = "ami-03054015e26069645" -> null
      - arn                                  = "arn:aws:ec2:ap-south-1:885596515371:instance/i-0719e36443ddc768f" -> null
      - associate_public_ip_address          = true -> null
      - availability_zone                    = "ap-south-1b" -> null
      - disable_api_stop                     = false -> null
      - disable_api_termination              = false -> null
      - ebs_optimized                        = false -> null
      - force_destroy                        = false -> null
      - get_password_data                    = false -> null
      - hibernation                          = false -> null
      - id                                   = "i-0719e36443ddc768f" -> null
      - instance_initiated_shutdown_behavior = "stop" -> null
      - instance_state                       = "running" -> null
      - instance_type                        = "t3.micro" -> null
```

```
      - volume_size           = 8 -> null
      - volume_type           = "gp3" -> null
        # (1 unchanged attribute hidden)
    }
}

Plan: 0 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  - instance_id   = "i-0719e36443ddc768f" -> null
  - instance_type = "t3.micro" -> null
  - public_ip     = "13.203.101.35" -> null

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

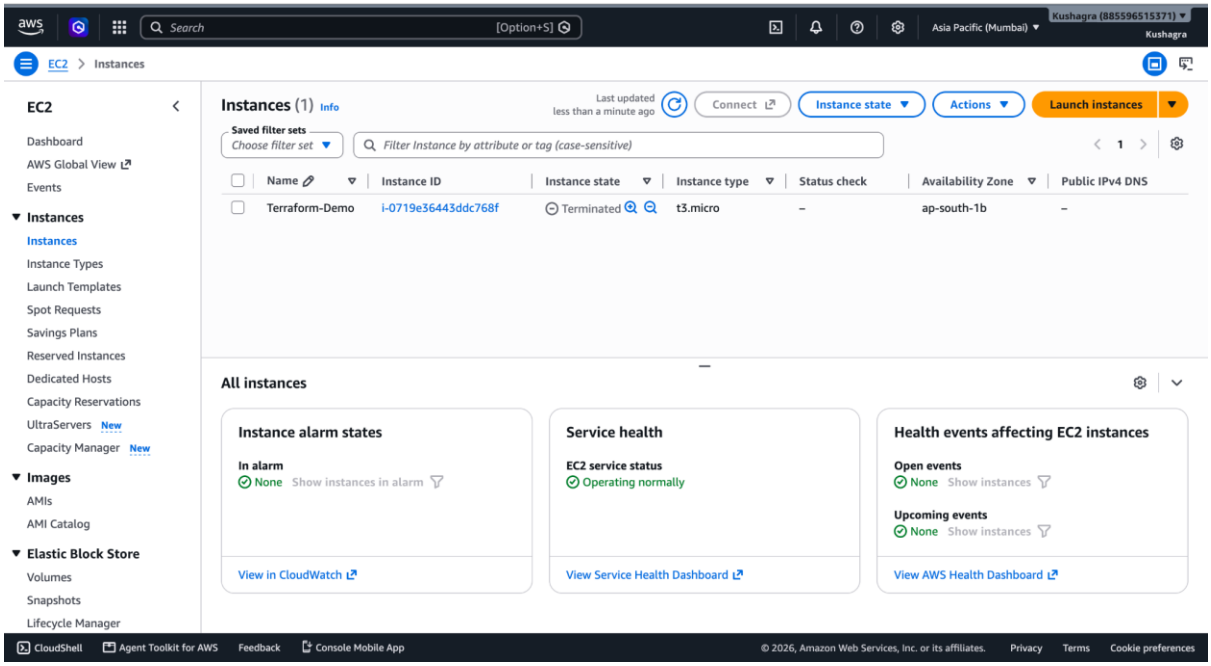
  Enter a value: yes

aws_instance.demo: Destroying... [id=i-0719e36443ddc768f]
aws_instance.demo: Still destroying... [id=i-0719e36443ddc768f, 00m10s elapsed]
aws_instance.demo: Still destroying... [id=i-0719e36443ddc768f, 00m20s elapsed]
aws_instance.demo: Still destroying... [id=i-0719e36443ddc768f, 00m30s elapsed]
aws_instance.demo: Destruction complete after 30s

Destroy complete! Resources: 1 destroyed.
```

20. Verify the Termination:

Verify in the AWS Management Console that the EC2 instance has been terminated.



Conclusion:

In this experiment, Terraform was used to provision and manage cloud infrastructure on Amazon Web Services (AWS) following the concept of Infrastructure as Code (IaC). An EC2 instance was created in the AWS Mumbai region by writing Terraform configuration files and executing commands such as terraform init, terraform validate, terraform plan, and terraform apply.

Resource details were then retrieved using Terraform outputs, and the infrastructure was tracked through the Terraform state file. Finally, the terraform destroy command was used to remove the created EC2 instance. The experiment demonstrated how Terraform simplifies

and automates cloud infrastructure management, making the deployment process more consistent, repeatable, scalable, and efficient.